

■ Master C++ & Data Structures & Algorithms (DSA) Syllabus

A Complete Roadmap from Language Fundamentals to Advanced Algorithmic Mastery

■ Syllabus Overview

This syllabus is designed as a definitive, production-grade guide to mastering **Data Structures & Algorithms (DSA)** along with **C++ (Basic, Intermediate, Advanced)**.

It is structured into **12 core modules**:

- **Module 0:** C++ Quick Revision & Mastery (Basic, Intermediate, Advanced)
 - **Module 1:** Arrays & Strings
 - **Module 2:** Linked Lists
 - **Module 3:** Stacks & Queues
 - **Module 4:** Recursion & Backtracking
 - **Module 5:** Sorting & Binary Search
 - **Module 6:** Trees & Binary Search Trees (BST)
 - **Module 7:** Heaps & Priority Queues
 - **Module 8:** Hashing & Bit Manipulation
 - **Module 9:** Graphs
 - **Module 10:** Dynamic Programming (DP)
 - **Module 11:** Advanced Data Structures & String Algorithms
-

■ Module 0: C++ Quick Revision & Mastery

Before diving into algorithmic problems, solidifying language constructs, memory models, and STL containers in C++ is essential.

0.1 Basic C++

- **Syntax & I/O:**
 - Structure of a C++ program (`#include <iostream>`, `main()`).
 - Fast I/O trick for competitive programming/DSA:

```
ios_base::sync_with_stdio(false);
cin.tie(NULL);
```

- **Data Types, Ranges & Precision:**

- Primitive types (`int`, `long`, `long long`, `float`, `double`, `char`, `bool`).
- Overflow/underflow awareness ($2^{31}-1$ vs $2^{63}-1$).

- **Control Flow:** Conditionals (`if`, `else if`, `switch`), Loops (`for`, `while`, `do-while`, range-based `for`).

- **Arrays & Strings:** C-style fixed arrays vs `std::string`, string manipulation (`length()`, `substr()`, `find()`, `append()`).

- **Functions & Parameter Passing:**

- Pass-by-Value vs Pass-by-Reference (`&`).
- Pass-by-Const-Reference (`const std::string& str`) to eliminate copy overhead.

- **Pointers & Memory Addresses:** Address-of operator (`&`), Dereference operator (`*`), Pointer arithmetic, Null pointers (`nullptr`).

0.2 Intermediate C++ & Standard Template Library (STL)

- **Sequence Containers:**

- `std::vector`: Dynamic array, amortized $O(1)$ `push_back`, capacity vs size, `reserve()`, `emplace_back()`.
- `std::deque`: Double-ended queue, $O(1)$ push/pop at both ends.
- `std::list`: Doubly linked list, $O(1)$ insertions/deletions anywhere given an iterator.

- **Associative Containers:**

- `std::set` / `std::map`: Self-balancing Red-Black Tree implementation, sorted keys, $O(\log N)$ search/insert/delete.
- `std::unordered_set` / `std::unordered_map`: Hash table implementation, average $O(1)$ search/insert/delete, worst-case $O(N)$ due to collisions.
- `std::pair` & `std::tuple`: Binding multiple data types.

- **Container Adaptors:**

- `std::stack`: LIFO operations (`push`, `pop`, `top`).
- `std::queue`: FIFO operations (`push`, `pop`, `front`).
- `std::priority_queue`: Max-heap by default (`top`, `push`, `pop`), Min-heap via `std::priority_queue<int, vector<int>, greater<int>>`.

- **Essential STL Algorithms:**

- `std::sort(begin, end)`: IntroSort ($O(N \log N)$).
- `std::binary_search`, `std::lower_bound`, `std::upper_bound`.
- `std::next_permutation`, `std::prev_permutation`.
- `std::min_element`, `std::max_element`, `std::accumulate`.

- **Dynamic Memory & Object-Oriented Programming (OOP):**

- Dynamic allocation (`new`, `delete`, `new[]`, `delete[]`).
- Classes vs Structs (default access modifiers: `private` vs `public`).
- Constructors (Default, Parameterized, Copy), Destructors.
- Virtual functions, Abstract classes, and Polymorphism.

0.3 Advanced C++

- **Smart Pointers (RAII):** `std::unique_ptr` (exclusive ownership), `std::shared_ptr` (reference-counted ownership), `std::weak_ptr` (preventing cyclic references).
- **Move Semantics & Rvalue References:** Rvalue reference (**T&&**), `std::move`, move constructors, avoiding deep copies.
- **Templates & Generic Programming:** Function templates, Class templates, Template specialization.
- **Lambda Expressions & Custom Comparators:**
 - Lambda syntax: `capture -> ret { body }`.
 - Custom comparators for sorting:

```
sort(vec.begin(), vec.end(), [](const Point& a, const Point& b) {  
    return a.x < b.x;  
});
```

- Custom comparators for `std::priority_queue`.
- **Custom Hash Functions:** Writing custom hashes for `std::unordered_map<pair<int,int>, int>` using XOR / `boost::hash_combine`.
- **Memory Alignment & Cache Locality:** Spatial & Temporal locality in array traversals vs linked data structures.

■ Module 1: Arrays & Strings

1.1 Core Concepts & Theory

- Memory layout of 1D and 2D arrays (Row-Major vs Column-Major ordering).
- Time and Space complexity of Array operations (Access $O(1)$, Search $O(N)$, Insertion/Deletion $O(N)$).
- String representation, immutability concepts in different languages, and character encodings (ASCII / UTF-8).

1.2 Theoretical Subtopics & Algorithmic Patterns

- **Two Pointers Technique:** Opposite direction (e.g., palindrome check, 2-sum) and Same direction (e.g., fast/slow pointers, removing duplicates).
- **Sliding Window:** Fixed-size window (e.g., max sum of subarray of size K) and Variable-size window (e.g., longest substring with K distinct characters).
- **Prefix Sum & Difference Array:** $O(1)$ range sum queries, $O(1)$ range updates using difference array.
- **Kadane's Algorithm:** Maximum subarray sum intuition, local vs global optimal state transition.
- **Dutch National Flag Algorithm:** 3-way partitioning algorithm ($O(N)$ time, $O(1)$ space).
- **2D Matrix Manipulation:** In-place matrix rotation, spiral traversal, matrix multiplication.

1.3 Must-Do Famous Problems

- **Two Sum** (*Array, Hash Table*) — LeetCode #1
- **Best Time to Buy and Sell Stock** (*Array, Dynamic Programming*) — LeetCode #121

- **Maximum Subarray (Kadane's Algorithm)** (*Array, DP*) — LeetCode #53
 - **3Sum** (*Two Pointers, Array, Sorting*) — LeetCode #15
 - **Container With Most Water** (*Two Pointers, Greedy*) — LeetCode #11
 - **Trapping Rain Water** (*Two Pointers, Stack, Array*) — LeetCode #42
 - **Find All Anagrams in a String** (*Sliding Window, Hash Table*) — LeetCode #438
 - **Subarray Sum Equals K** (*Prefix Sum, Hash Table*) — LeetCode #560
 - **Spiral Matrix** (*2D Array, Simulation*) — LeetCode #54
 - **Minimum Window Substring** (*Sliding Window, Hash Table*) — LeetCode #76
-

■ Module 2: Linked Lists

2.1 Core Concepts & Theory

- Linked List Node structure (Data field, Next/Prev pointer fields).
- Memory allocation (Non-contiguous node distribution vs Array contiguous memory).
- Variations: Singly Linked List, Doubly Linked List, Circular Linked List.

2.2 Theoretical Subtopics & Algorithmic Patterns

- **Pointer Manipulation:** In-place node insertion, deletion, and pointer updating without loss of references.
- **Fast & Slow Pointers (Floyd's Cycle Detection):** Mathematical proof of cycle detection ($2k - k = n \cdot C$), finding cycle entry node.
- **Linked List Reversal:** Iterative 3-pointer method (**prev**, **curr**, **next**) and Recursive reversal.
- **Dummy Node Pattern:** Eliminating edge cases for head updates.
- **Merge Sort on Linked Lists:** Finding middle using slow/fast pointers, dividing list, merging sorted lists ($O(N \log N)$ time, $O(1)$ auxiliary space).
- **Cache Eviction Architectures:** Combined Hash Map + Doubly Linked List for LRU Cache ($O(1)$ get and put).

2.3 Must-Do Famous Problems

- **Reverse Linked List** (*Iterative & Recursive*) — LeetCode #206
- **Middle of the Linked List** (*Fast & Slow Pointers*) — LeetCode #876
- **Linked List Cycle I & II** (*Floyd's Cycle Detection*) — LeetCode #141 / #142
- **Merge Two Sorted Lists** (*Two Pointers, Recursion*) — LeetCode #21
- **Remove Nth Node From End of List** (*Two Pointers*) — LeetCode #19
- **Reorder List** (*Middle split + Reverse + Merge*) — LeetCode #143
- **Add Two Numbers** (*Math, Linked List*) — LeetCode #2
- **Merge k Sorted Lists** (*Heap / Divide & Conquer*) — LeetCode #23
- **LRU Cache** (*Hash Table, Doubly Linked List*) — LeetCode #146
- **LFU Cache** (*Hash Table, Doubly Linked List, Frequency Map*) — LeetCode #460

■ Module 3: Stacks & Queues

3.1 Core Concepts & Theory

- **Stack:** LIFO (Last In First Out) paradigm, Function Call Stack execution model.
- **Queue:** FIFO (First In First Out) paradigm, Task scheduling queues.
- Array-based vs Linked List-based implementations, Array overflow/underflow handling.

3.2 Theoretical Subtopics & Algorithmic Patterns

- **Monotonic Stack:** Next Greater Element (NGE), Next Smaller Element (NSE), Previous Greater/Smaller Element.
- **Monotonic Queue:** Sliding Window Maximum tracking using Deque ($O(N)$ amortized time).
- **Expression Evaluation & Parsing:** Infix to Postfix (Shunting-yard Algorithm), Postfix Evaluation using Stack.
- **Min Stack / Max Stack:** Auxiliary stack technique vs $O(1)$ space math encoding ($2 * val - minVal$).

3.3 Must-Do Famous Problems

- **Valid Parentheses** (*Stack*) — LeetCode #20
- **Min Stack** (*Design, Stack*) — LeetCode #155
- **Evaluate Reverse Polish Notation** (*Stack, Math*) — LeetCode #150
- **Daily Temperatures** (*Monotonic Stack*) — LeetCode #739
- **Next Greater Element I & II** (*Monotonic Stack*) — LeetCode #496 / #503
- **Largest Rectangle in Histogram** (*Monotonic Stack*) — LeetCode #84
- **Sliding Window Maximum** (*Monotonic Queue, Deque*) — LeetCode #239
- **Online Stock Span** (*Monotonic Stack*) — LeetCode #901
- **Implement Queue using Stacks** (*Design, Stack*) — LeetCode #232
- **Basic Calculator I & II** (*Stack, Math, String Parsing*) — LeetCode #224 / #227

■ Module 4: Recursion & Backtracking

4.1 Core Concepts & Theory

- Recursion tree visualization, Base Case, Recursive State Transition, Call Stack Depth & Stack Overflow.
- Backtracking paradigm: Explore, Choose, Un-choose (State Reset).

4.2 Theoretical Subtopics & Algorithmic Patterns

- **State Space Tree Search:** Systematic exploration of search space.
- **Subsets / Power Set (Include/Exclude Pattern):** Decision tree branches for each element.

- **Combinations & Permutations:** Handling duplicate elements via sorting and frequency maps / boolean visited arrays.
- **Pruning:** Early termination of invalid recursive paths to avoid TLE.
- **Constraint Satisfaction Problems:** Grid backtracking, N-Queens placement validation, Sudoku cell constraints.

4.3 Must-Do Famous Problems

- **Subsets I & II** (*Backtracking, Bit Manipulation*) — LeetCode #78 / #90
 - **Permutations I & II** (*Backtracking*) — LeetCode #46 / #47
 - **Combination Sum I & II** (*Backtracking*) — LeetCode #39 / #40
 - **Letter Combinations of a Phone Number** (*Backtracking, String*) — LeetCode #17
 - **Word Search** (*Matrix Backtracking, DFS*) — LeetCode #79
 - **Palindrome Partitioning** (*Backtracking, DP*) — LeetCode #131
 - **N-Queens** (*Backtracking, Constraint Satisfaction*) — LeetCode #51
 - **Sudoku Solver** (*Backtracking, Constraint Satisfaction*) — LeetCode #37
-

■ Module 5: Sorting & Searching (Binary Search)

5.1 Core Concepts & Theory

- **Sorting Categories:** Comparison-based vs Non-comparison-based, In-place vs Out-of-place, Stable vs Unstable sorts.
- **Binary Search Paradigm:** Reducing search space by half at each step ($O(\log N)$ time), Condition for Monotonicity.

5.2 Theoretical Subtopics & Algorithmic Patterns

- **Sorting Algorithms Breakdown:**
 - Insertion Sort, Selection Sort, Bubble Sort ($O(N^2)$).
 - Merge Sort ($O(N \log N)$ Stable), Quick Sort ($O(N \log N)$ Average, Pivot Selection strategies).
 - QuickSelect Algorithm ($O(N)$ average time for Kth element).
 - Counting Sort, Radix Sort ($O(N + K)$ non-comparison).
- **Binary Search Variants:**
 - Standard BS, Order-Agnostic BS.
 - Lower Bound (**first element $\geq$ target**), Upper Bound (**first element $>$ target**).
 - Binary search on rotated sorted arrays (identifying sorted half).
- **Binary Search on Answer Space:** Monotonic predicate functions $f(x)$ to $\{ \text{True}, \text{False} \}$, defining search range **[low, high]**.

5.3 Must-Do Famous Problems

- **Binary Search** (*Search, Array*) — LeetCode #704
 - **Search in Rotated Sorted Array I & II** (*Binary Search*) — LeetCode #33 / #81
 - **Find First and Last Position of Element in Sorted Array** (*Lower/Upper Bound*) — LeetCode #34
 - **Kth Largest Element in an Array** (*QuickSelect / Heap*) — LeetCode #215
 - **Search a 2D Matrix I & II** (*Binary Search*) — LeetCode #74 / #240
 - **Find Minimum in Rotated Sorted Array** (*Binary Search*) — LeetCode #154
 - **Koko Eating Bananas** (*Binary Search on Answer*) — LeetCode #875
 - **Capacity To Ship Packages Within D Days** (*Binary Search on Answer*) — LeetCode #1011
 - **Aggressive Cows / Book Allocation Problem** (*Binary Search on Answer*) — Standard Pattern
 - **Median of Two Sorted Arrays** (*Advanced Binary Search*) — LeetCode #4
-

■ Module 6: Trees & Binary Search Trees (BST)

6.1 Core Concepts & Theory

- Tree Definitions: Root, Child, Parent, Leaf, Depth, Height, Ancestor, Subtree.
- Binary Tree Types: Full, Complete, Perfect, Balanced, Degenerate.
- Binary Search Tree Invariant: $\text{Left Subtree} < \text{Node} < \text{Right Subtree}$.

6.2 Theoretical Subtopics & Algorithmic Patterns

- **Tree Traversals:**
 - Depth-First: Inorder (Left-Node-Right), Preorder (Node-Left-Right), Postorder (Left-Right-Node) via Recursion & Iterative Stacks.
 - Breadth-First: Level-Order Traversal using Queue.
- **Lowest Common Ancestor (LCA):** Top-down vs Bottom-up search logic for BT and BST.
- **Tree Construction:** Reconstructing tree from (Preorder + Inorder) or (Postorder + Inorder).
- **Diameter & Path Sums:** Global maximum update during postorder traversal.
- **BST Operations:** Search, Insert, Delete node (3 cases: Leaf, 1 Child, 2 Children via Inorder Successor/Predecessor).
- **Self-Balancing Concepts:** AVL Rotations (LL, RR, LR, RL), Red-Black tree properties (conceptual).

6.3 Must-Do Famous Problems

- **Maximum Depth of Binary Tree** (*DFS/BFS*) — LeetCode #104
- **Invert / Flip Binary Tree** (*Recursion*) — LeetCode #226
- **Diameter of Binary Tree** (*DFS, Global Max*) — LeetCode #543
- **Binary Tree Level Order Traversal** (*BFS, Queue*) — LeetCode #102
- **Lowest Common Ancestor of a Binary Tree** (*Recursion*) — LeetCode #236

- **Validate Binary Search Tree** (*BST Invariant, Range Check*) — LeetCode #98
 - **Kth Smallest Element in a BST** (*Inorder Traversal*) — LeetCode #230
 - **Construct Binary Tree from Preorder and Inorder Traversal** (*Recursion, Hash Table*) — LeetCode #105
 - **Binary Tree Maximum Path Sum** (*Postorder DFS, Dynamic Programming*) — LeetCode #124
 - **Serialize and Deserialize Binary Tree** (*Design, BFS/DFS*) — LeetCode #297
-

■ ■ Module 7: Heaps & Priority Queues

7.1 Core Concepts & Theory

- Complete Binary Tree representation in Array (Parent at $i/2$, Left Child at $2i$, Right Child at $2i+1$).
- Max-Heap vs Min-Heap properties.
- Building Heap (**Heapify**): $O(N)$ time complexity proof vs $O(N \log N)$ successive insertion.

7.2 Theoretical Subtopics & Algorithmic Patterns

- **Heap Operations:** **push** ($O(\log N)$), **pop** ($O(\log N)$), **top** ($O(1)$).
- **Top-K Elements Pattern:** Maintaining a Min-Heap of size K for K largest elements.
- **K-Way Merge Pattern:** Merging K sorted streams using Priority Queue.
- **Two-Heaps Pattern:** Min-Heap + Max-Heap combination to track dynamic median in $O(1)$ time.

7.3 Must-Do Famous Problems

- **Kth Largest Element in a Stream** (*Min-Heap*) — LeetCode #703
 - **Top K Frequent Elements** (*Min-Heap / Bucket Sort*) — LeetCode #347
 - **K Closest Points to Origin** (*Max-Heap / QuickSelect*) — LeetCode #973
 - **Task Scheduler** (*Max-Heap, Greedy, Queue*) — LeetCode #621
 - **Reorganize String** (*Max-Heap, Greedy*) — LeetCode #767
 - **Find Median from Data Stream** (*Two Heaps: Min & Max*) — LeetCode #295
 - **Merge k Sorted Lists** (*Priority Queue / Divide & Conquer*) — LeetCode #23
 - **Smallest Range Covering Elements from K Lists** (*Min-Heap, Sliding Window*) — LeetCode #632
-

■ Module 8: Hashing & Bit Manipulation

8.1 Core Concepts & Theory

- **Hashing:** Hash Functions, Load Factor ($\alpha = N/M$), Collision Resolution (Separate Chaining using LL/Vectors vs Open Addressing: Linear/Quadratic Probing, Double Hashing).
- **Bit Manipulation:** Binary representations, Two's Complement, Bitwise operators ($\&$, $|$, $\wedge$, $\sim$, $\ll$, $\gg$).

8.2 Theoretical Subtopics & Algorithmic Patterns

- **Bit Manipulation Core Formulas:**

- Check i -th bit set: $(n \& (1 \ll i)) \neq 0$
- Set i -th bit: $n \mid (1 \ll i)$
- Clear i -th bit: $n \& \sim(1 \ll i)$
- Toggle i -th bit: $n \wedge (1 \ll i)$
- Unset lowest set bit: $n \& (n - 1)$ (Kernighan's Algorithm)
- Isolate lowest set bit: $n \& (-n)$
- Check Power of 2: $(n > 0) \&\& ((n \& (n - 1)) == 0)$

- **Bitmasking:** Representing sets as integers for state compression (2^N states).

8.3 Must-Do Famous Problems

- **Single Number I, II, & III** (*XOR Bitwise Logic*) — LeetCode #136 / #137 / #260
- **Number of 1 Bits (Hamming Weight)** (*Bit Operations*) — LeetCode #191
- **Counting Bits** (*Bitwise DP*) — LeetCode #338
- **Reverse Bits** (*Bit Manipulation*) — LeetCode #190
- **Group Anagrams** (*Hash Map, Categorization*) — LeetCode #49
- **Longest Consecutive Sequence** (*Unordered Set, $O(N)$ Search*) — LeetCode #128
- **Subarray Sum Equals K** (*Prefix Sum + Hash Map*) — LeetCode #560
- **Bitwise AND of Numbers Range** (*Bit Shift Logic*) — LeetCode #201
- **Subsets using Bitmasking** (*Combinatorics, Bitmask*) — LeetCode #78

■ Module 9: Graphs

9.1 Core Concepts & Theory

- Graph Representations: Adjacency Matrix ($O(V^2)$ space) vs Adjacency List ($O(V + E)$ space).
- Terminology: Directed, Undirected, Weighted, Unweighted, Cyclic, Acyclic (DAG), Connected Components, Bipartite.

9.2 Theoretical Subtopics & Algorithmic Patterns

- **Graph Traversals:**

- **BFS:** Shortest path in unweighted graphs using Queue ($O(V + E)$).
- **DFS:** Recursive exploration using Call Stack ($O(V + E)$).

- **Cycle Detection:**

- Undirected Graph: BFS/DFS (visited parent check) or DSU.
- Directed Graph: DFS (In-degree / Recursion Stack tracking) or Kahn's Algorithm.

- **Topological Sort (DAG):**

- Kahn's Algorithm (BFS with In-degree array).
- DFS with Stack.
- **Disjoint Set Union (DSU / Union-Find):**
 - Path Compression & Union by Rank/Size ($O(\alpha(N))$ amortized time per operation).
- **Shortest Path Algorithms:**
 - **Dijkstra's Algorithm:** Non-negative weights using Priority Queue ($O((V + E) \log V)$).
 - **Bellman-Ford Algorithm:** Single-source shortest paths with negative weights & negative cycle detection ($O(V \cdot E)$).
 - **Floyd-Warshall Algorithm:** All-pairs shortest paths using 3D/2D DP ($O(V^3)$).
- **Minimum Spanning Tree (MST):**
 - **Kruskal's Algorithm:** Greedy Edge sorting + DSU ($O(E \log E)$).
 - **Prim's Algorithm:** Greedy Vertex expansion + Priority Queue ($O(E \log V)$).
- **Advanced Graph Theory:** Tarjan's Algorithm for Bridges & Articulation Points (Low-link values).

9.3 Must-Do Famous Problems

- **Number of Islands** (*Grid BFS/DFS, Connected Components*) — LeetCode #200
- **Clone Graph** (*Graph Hash Map, BFS/DFS*) — LeetCode #133
- **Course Schedule I & II** (*Topological Sort, Cycle Detection*) — LeetCode #207 / #210
- **Is Graph Bipartite?** (*Coloring BFS/DFS*) — LeetCode #785
- **Word Ladder** (*Shortest Path BFS*) — LeetCode #127
- **Network Delay Time** (*Dijkstra's Algorithm*) — LeetCode #743
- **Cheapest Flights Within K Stops** (*Bellman-Ford / Modified Dijkstra*) — LeetCode #787
- **Redundant Connection** (*DSU / Cycle Detection*) — LeetCode #684
- **Min Cost to Connect All Points** (*Kruskal's / Prim's MST*) — LeetCode #1584
- **Critical Connections in a Network (Bridges)** (*Tarjan's Algorithm*) — LeetCode #1192

■ Module 10: Dynamic Programming (DP)

10.1 Core Concepts & Theory

- Core Requirements: **Overlapping Subproblems & Optimal Substructure.**
- Approaches:
 - **Top-Down (Memoization):** Recursive exploration + Cache lookup.
 - **Bottom-Up (Tabulation):** Iterative DP table population.
 - **Space Optimization:** Reducing state tables from $O(N^2)$ to $O(N)$ or $O(N)$ to $O(1)$ by identifying necessary prior states.

10.2 Theoretical Subtopics & Categories

- **1D DP:** Fibonacci sequence pattern, House Robber, Climbing Stairs.
- **2D / Grid DP:** Unique Paths, Minimum Path Sum, Matrix traversals.
- **Knapsack Patterns:**
 - **0/1 Knapsack:** Item inclusion/exclusion once (Subset Sum, Equal Partition).
 - **Unbounded Knapsack:** Infinite supply of items (Coin Change, Rod Cutting).
- **Longest Common Subsequence (LCS):** String matching DP transitions, Edit Distance, Shortest Common Supersequence.
- **Longest Increasing Subsequence (LIS):** $O(N^2)$ DP formulation vs $O(N \log N)$ Binary Search patience sorting formulation.
- **Interval DP / Matrix Chain Multiplication (MCM):** Subsegment split optimizations.
- **Tree DP:** State transitions across subtrees (Diameter, Max Path Sum).
- **Bitmask DP:** State compression over set elements.

10.3 Must-Do Famous Problems

- **Climbing Stairs** (*1D DP*) — LeetCode #70
- **House Robber I & II** (*1D DP, Circular*) — LeetCode #198 / #213
- **Coin Change I & II** (*Unbounded Knapsack*) — LeetCode #322 / #518
- **Partition Equal Subset Sum** (*0/1 Knapsack*) — LeetCode #416
- **Longest Common Subsequence** (*2D String DP*) — LeetCode #1143
- **Edit Distance** (*2D String DP*) — LeetCode #72
- **Longest Increasing Subsequence** (*LIS, $O(N \log N)$ Binary Search*) — LeetCode #300
- **Word Break** (*DP / Trie*) — LeetCode #139
- **Unique Paths I & II** (*Grid DP*) — LeetCode #62 / #63
- **Maximum Product Subarray** (*1D DP, Min/Max Tracking*) — LeetCode #152
- **Palindrome Partitioning II** (*String DP, Min Cut*) — LeetCode #132
- **Burst Balloons** (*Interval DP / MCM*) — LeetCode #312

■ Module 11: Advanced Data Structures & String Algorithms

11.1 Core Concepts & Theory

- Advanced String Searching and Range Query Structures designed for scalable query optimization.

11.2 Theoretical Subtopics & Patterns

- **Trie (Prefix Tree):** Node structure with child pointers array `children[26]` and `isEndOfWord` flag. $O(L)$ insertion/search where L is word length.
- **Segment Tree:** Complete binary tree for range queries (Range Sum, Range Min/Max) and point updates in $O(\log N)$ time. Build tree in $O(N)$.

- **Binary Indexed Tree (Fenwick Tree / BIT):** Prefix sum range queries and point updates using bitwise low-bit manipulation (`idx & (-idx)`) with $O(N)$ space and $O(\log N)$ query time.
- **KMP (Knuth-Morris-Pratt) Algorithm:** String pattern searching in $O(N + M)$ time using Longest Prefix Suffix (LPS) array.

11.3 Must-Do Famous Problems

- **Implement Trie (Prefix Tree)** (*Trie Design*) — LeetCode #208
 - **Design Add and Search Words Data Structure** (*Trie, Backtracking*) — LeetCode #211
 - **Maximum XOR of Two Numbers in an Array** (*Bitwise Trie*) — LeetCode #421
 - **Range Sum Query - Mutable** (*Segment Tree / Fenwick Tree*) — LeetCode #307
 - **Find the Index of the First Occurrence in a String (KMP)** (*KMP Algorithm / LPS*) — LeetCode #28
 - **Shortest Palindrome** (*KMP Algorithm / String*) — LeetCode #214
-

■ Suggested Learning Schedule & Progress Checklist

- ☐ **Weeks 1–2:** C++ Basics, STL Mastery, & Pointers/Memory Management (Module 0)
- ☐ **Weeks 3–4:** Arrays, Two Pointers, & Sliding Window (Module 1)
- ☐ **Weeks 5–6:** Linked Lists & Monotonic Stacks/Queues (Modules 2 & 3)
- ☐ **Weeks 7–8:** Recursion, Backtracking, & Binary Search (Modules 4 & 5)
- ☐ **Weeks 9–10:** Trees, BSTs, & Priority Queues (Modules 6 & 7)
- ☐ **Weeks 11–12:** Hashing, Bit Manipulation, & Graph Fundamentals (Modules 8 & 9)
- ☐ **Weeks 13–14:** Advanced Graphs & Dynamic Programming (Modules 9 & 10)
- ☐ **Weeks 15–16:** Advanced Data Structures & Interview Revision (Module 11)